Diffusion 프로젝트를 위한 핵심 개념과 수식

CFG · Guided / Progressive Distillation
Consistency · NFE · EMA · FID

FastGen handoff 및 실제 overlay 코드 기반

2026년 10월 7일

이 문서의 목적 Handoff를 처음 읽는 팀원이 생성 모델의 기본 원리부터 현재 1/4-NFE 모델의 학습·추론·평가까지 이해하도록 정리한다. 개념 설명과 현재 구현 여부를 구분한다.

본 프로젝트는 151개 Pokémon 클래스의 64×64 RGB 이미지를 생성한다. 현재 구현은 **fixed-CFG guidance transfer + progressive distillation**이다. Consistency distillation은 향후 baseline이며 아직 구현되지 않았다.

읽는 순서: 1–3절로 diffusion의 입출력을 이해하고, 4–6절로 CFG와 압축을 익힌다. 7절은 다음 연구 방향, 8–10절은 코드·학습·평가와의 연결이다. 마지막에는 프로젝트 결과, 용어 사전, 확인 문제와 출처가 있다.

범위: 과거 실험은 `TEAM_HANDOFF.md`와 code ZIP의 2026-09-30 완료 기록을 기준으로 한다. 후속 요청으로 실행 환경 복원과 제출 모델의 공식 FID 재평가를 수행했으며 그 확인값도 명시한다. 기존 teacher/student 학습 continuation은 수행하지 않았다. 논문 일반론을 프로젝트 구현 완료로 해석하지 않는다.

LaTeX 원본: `DIFFUSION_CONCEPTS.tex`. XeLaTeX와 한국어 글꼴로 컴파일한다. 수식의 guidance scale은 g, noise 표준편차는 σ로 구분한다.

Contents

1 문제를 이해하기 위한 기본 개념 **4**
1.1 생성 모델과 조건부 생성 . . . 4
1.2 학습과 추론, teacher와 student . . . 4
1.3 표기와 tensor shape . . . 4

2 Diffusion: noise를 더하는 학습, noise를 빼는 생성 **4**
2.1 Forward process . . . 4
2.2 Reverse process와 denoising . . . 5
2.3 Epsilon MSE와 SNR . . . 5

3 Prediction parameterization과 sampler **5**
3.1 Epsilon, clean image, velocity의 차이 . . . 5
3.2 DDPM과 DDIM . . . 5
3.3 Timestep grid와 clipping . . . 6

4 Classifier-Free Guidance: CFG **6**
4.1 왜 guidance가 필요한가 . . . 6
4.2 한 network로 conditional / unconditional 학습 . . . 6
4.3 CFG 식과 scale 해석 . . . 6
4.4 계산량과 구현 위치 . . . 6

5 Distillation: teacher의 동작을 student에 전달 **6**
5.1 Knowledge distillation의 의미 . . . 6
5.2 Guidance distillation과 fixed / variable guidance . . . 7
5.3 현재 guidance-transfer loss . . . 7

6 Progressive distillation: 두 step을 한 step으로 **7**
6.1 직관과 stage 구성 . . . 7
6.2 Merged target 유도 . . . 7
6.3 왜 마지막 $2 \to 1$이 어렵나 . . . 8

7 Consistency models와 consistency distillation **8**
7.1 현재 구현과의 경계 . . . 8
7.2 Score와 probability-flow ODE . . . 8
7.3 Adjacent-pair loss와 boundary condition . . . 8
7.4 방법 비교 . . . 9

8 U-Net과 conditioning: 수식이 실행되는 구조 **9**
8.1 Encoder, decoder, skip connection . . . 9
8.2 Residual block, normalization, attention . . . 9
8.3 Embedding과 parameter count . . . 9

9 학습 안정성과 재현성에 필요한 개념 **10**
9.1 Optimizer, learning rate, update budget . . . 10
9.2 EMA: weight의 지수 이동평균 . . . 10
9.3 AMP, bf16, FP32, clipping, VRAM . . . 10
9.4 Checkpoint, export, resume . . . 10

10 NFE, FID, holdout: 결과를 올바르게 읽기 **11**
10.1 Steps와 NFE의 차이 . . . 11
10.2 FID의 의미와 한계 . . . 11
10.3 Training-only holdout과 official validation . . . 11
10.4 Ablation과 seed . . . 11

11 개념을 현재 프로젝트에 연결하기 **11**
11.1 최신 결과와 현재 병목 . . . 11
11.2 각 실험의 질문과 연결 관계 . . . 12
11.3 코드 찾아보기 . . . 12
11.4 One-NFE 한 번의 생성 따라가기 . . . 12
11.5 다음 연구를 판단하는 질문 . . . 13

12 빠른 용어 사전과 자주 생기는 혼동 **13**
12.1 이해 확인 문제 . . . 13

13 출처와 재생성 방법 **14**
13.1 프로젝트의 1차 근거 . . . 14
13.2 LaTeX에서 PDF 생성 . . . 14

1 문제를 이해하기 위한 기본 개념

1.1 생성 모델과 조건부 생성

분류 모델이 이미지에서 클래스 c를 찾는다면, 생성 모델은 noise z와 조건 c에서 이미지 x를 만든다. 조건부 생성은 데이터 분포 $p(x \mid c)$를 학습하는 문제다. 같은 클래스도 pose, 배경, 그림체가 다르므로 하나의 정답 이미지로 수렴시키는 회귀 문제와 다르다.

$$z \sim \mathcal{N}(0, I), \qquad x = G_\theta(z, c).$$

θ는 학습되는 parameter이고 z는 샘플마다 달라지는 무작위 입력이다. Class embedding은 정수 ID를 학습 가능한 vector로 변환한다. 현재 embedding은 151개 실제 클래스와 1개 null token으로 구성된다.

1.2 학습과 추론, teacher와 student

학습(training)에서는 gradient로 parameter를 업데이트한다. **추론(inference)**에서는 고정 weight로 이미지를 생성한다. Teacher는 목표를 제공하는 모델, student는 그 목표를 적은 계산으로 근사하는 모델이다. Teacher가 반드시 parameter 수가 더 커야 하는 것은 아니다. 현재 두 모델은 동일한 48.88M U-Net이며 sampling 횟수가 다르다.

1.3 표기와 tensor shape

표기	의미
x_0	깨끗한 실제 이미지. Discrete DDPM 표기의 출발점.
x_t	Noise level t의 이미지; shape $[B, 3, 64, 64]$.
ϵ	주입한 표준 Gaussian noise, $\mathcal{N}(0, I)$.
$\hat{\epsilon}_\theta, \hat{x}_{0,\theta}, \hat{v}_\theta$	Network가 예측하는 noise, clean image, velocity.
t, c	Batch별 timestep과 class ID; shape $[B]$.
$\beta_t, \bar{\alpha}_t$	Noise 증가량과 누적 신호 보존 비율.
a_t, σ_t	각각 $\sqrt{\bar{\alpha}_t}$, $\sqrt{1-\bar{\alpha}_t}$.
g	CFG guidance scale. σ_t와 다른 변수.

Pixel-space vs latent-space 현재 모델은 RGB pixel에 직접 noise를 더한다. Latent diffusion은 encoder가 만든 압축 공간에서 diffusion을 수행한다. 현재에는 VAE, latent decoder, 별도 refiner가 없다.

2 Diffusion: noise를 더하는 학습, noise를 빼는 생성

2.1 Forward process

실제 이미지에 조금씩 noise를 더하면 마지막에는 거의 Gaussian noise가 된다. DDPM의 전방 과정은 다음처럼 쓴다 [1].

$$q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\, x_{t-1}, \beta_t I), \tag{1}$$

$$\alpha_t = 1 - \beta_t, \qquad \bar{\alpha}_t = \prod_{i=0}^{t} \alpha_i, \tag{2}$$

$$x_t = a_t x_0 + \sigma_t \epsilon, \quad a_t^2 + \sigma_t^2 = 1. \tag{3}$$

마지막 식 덕분에 학습할 때 $0 \to 1 \to \cdots \to t$를 모두 실행할 필요가 없다. t 하나와 noise 하나를 뽑아 곧바로 x_t를 만들 수 있다. 코드에서는 0부터 999까지의 index를 사용하며 index 0에도 작은 noise가 남는다. 정확한 clean endpoint는 별도로 $t = -1$, $a = 1$, $\sigma = 0$으로 표현한다.

2.2 Reverse process와 denoising

Network는 x_t, t, c를 보고 noise를 예측한다. 이를 clean-image 추정으로 바꾸면

$$\hat{x}_0 = \frac{x_t - \sigma_t \hat{\epsilon}_\theta(x_t, t, c)}{a_t}. \tag{4}$$

추정치를 이용해 더 낮은 noise level로 이동하고 이를 반복하면 이미지가 된다. 각 단계에서 완벽한 원본을 찾는 것은 아니며 network의 오차와 sampler의 이동 규칙이 최종 품질에 영향을 준다.

2.3 Epsilon MSE와 SNR

현재 teacher의 학습 목표는 주입한 noise와 예측한 noise의 평균제곱오차다.

$$L_\epsilon = \mathbb{E}_{x_0,t,\epsilon,c'}\left[\|\hat{\epsilon}_\theta(x_t, t, c') - \epsilon\|^2\right], \qquad \mathrm{SNR}(t) = \frac{a_t^2}{\sigma_t^2}.$$

SNR(signal-to-noise ratio)은 noise에 비해 신호가 얼마나 큰지 나타낸다. 낮은 noise에서는 SNR이 크고 높은 noise에서는 작다. 식 (4)로부터 clean prediction의 오차는 noise 예측 오차의 σ_t/a_t배가 된다. 따라서 high-noise에서 작은 epsilon 오차가 큰 image 오차로 확대될 수 있다.

현재 설정: 1,000 timesteps, linear $\beta : 10^{-4} \to 0.02$. 이것은 **diffusion noise schedule**이며 optimizer의 learning-rate schedule과 독립이다. 낮은 training loss만으로 좋은 생성 품질을 보장하지 않는다.

3 Prediction parameterization과 sampler

3.1 Epsilon, clean image, velocity의 차이

같은 denoising 문제도 network의 출력 대상을 다르게 정할 수 있다.

출력	학습 대상	Clean prediction 복원
Epsilon	주입 noise ϵ	$(x_t - \sigma_t\hat{\epsilon})/a_t$
x_0	Clean image	$\hat{x}_0$ 직접 사용
Velocity v	$a_t\epsilon - \sigma_t x_0$	$a_t x_t - \sigma_t \hat{v}$

다음은 두 변수를 회전한 형태다. 직접 대입하면 역변환을 확인할 수 있다.

$$v = a_t\epsilon - \sigma_t x_0, \tag{5}$$

$$x_0 = a_t x_t - \sigma_t v, \qquad \epsilon = \sigma_t x_t + a_t v. \tag{6}$$

출력 shape가 같아도 의미는 다르므로 epsilon checkpoint를 그대로 v predictor로 해석하면 안 된다. 현재 첫 student는 teacher의 feature weight를 복사하되 출력 head를 초기화하고 v-prediction으로 학습한다. 여기서 velocity는 image의 물리적 이동 속도를 뜻하지 않는다.

3.2 DDPM과 DDIM

DDPM sampling은 일반적으로 역과정에서 추가 noise를 뽑는 stochastic update를 사용한다. **DDIM**은 같은 noise predictor를 이용해 다른 sampling 경로를 구성하며 $\eta = 0$에서는 결정적이다 [2]. 현재 distillation은 deterministic DDIM transition을 사용한다.

현재 시작 level t에서 다음 level $u < t$로 이동하는 식은

$$\hat{\epsilon} = (x_t - a_t\hat{x}_0)/\sigma_t, \tag{7}$$

$$x_u = a_u\hat{x}_0 + \sigma_u\hat{\epsilon}. \tag{8}$$

동일한 초기 noise, condition, weight, 계산 환경에서 같은 경로를 얻는다는 의미다. DDPM과 DDIM의 seed가 같아도 DDPM에는 추가 noise가 있으므로 전체 trajectory가 같지는 않다.

3.3 Timestep grid와 clipping

Sampling grid는 학습한 1,000개 noise level 중 어떤 순서와 간격으로 방문할지 정한다. 현재는 999부터 0까지 반올림한 32-node grid를 시작으로 매 stage에서 node를 절반씩 사용한다. 4-step weight에서 임의로 3 node를 골랐다고 3-step 품질이 검증되는 것은 아니다.

Clipping은 예측값을 범위 안으로 제한하는 연산이다. Teacher의 guided clean estimate는 $[-1, 1]$로 clip한다. Progressive merged target과 중간 student prediction은 clip하지 않는다. 최종 출력 RGB는 clip한다. 이 구분은 6절의 endpoint 보존과 연결된다.

4 Classifier-Free Guidance: CFG

4.1 왜 guidance가 필요한가

Class condition을 주어도 network가 이를 약하게 사용할 수 있다. CFG는 condition을 넣은 예측과 빼고 한 예측의 차이를 강조한다 [3]. 별도 image classifier의 gradient를 쓰는 classifier guidance와 구분한다.

4.2 한 network로 conditional / unconditional 학습

Training 중 확률 p_{drop}로 실제 class를 null token으로 바꾼다. 같은 parameter가 두 동작을 배운다. 현재 $p_{\text{drop}} = 0.1$, null ID는 151이다. Residual network dropout과 class dropout은 별개다.

$$\hat{\epsilon}_c = \epsilon_\theta(x_t, t, c), \qquad \hat{\epsilon}_\varnothing = \epsilon_\theta(x_t, t, \varnothing).$$

4.3 CFG 식과 scale 해석

$$\boxed{\hat{\epsilon}_{\text{CFG}} = \hat{\epsilon}_\varnothing + g(\hat{\epsilon}_c - \hat{\epsilon}_\varnothing)} \tag{9}$$

- $g = 0$: unconditional prediction.
- $g = 1$: ordinary class-conditional prediction.
- $g > 1$: conditional 방향을 extrapolate한다. 현재 fixed scale은 2다.

한 성분에서 unconditional 값이 0.2, conditional 값이 0.5라면 $g = 2$의 guided 값은 $0.2 + 2(0.5 - 0.2) = 0.8$이다. 이는 noise prediction 성분의 계산 예이며 이미지 brightness가 직접 0.8이 된다는 뜻은 아니다.

어떤 논문은 $(1 + w)\epsilon_c - w\epsilon_\varnothing$로 쓰므로 현재 표기와는 $g = 1 + w$ 관계다. 표기를 확인하지 않고 scale 수치를 옮기면 다른 guidance가 된다. 높은 scale은 condition 강조에 도움이 될 수 있지만 다양성·포화·왜곡과 trade-off가 있어 holdout에서 선택해야 한다.

4.4 계산량과 구현 위치

현재 CFG=2 teacher는 conditional과 unconditional backbone을 각각 호출한다. 32 denoising steps면 64 NFE다. Scale 0/1은 필요한 branch만 사용할 수 있다. Batched CFG로 한 API 호출에 합치더라도 두 branch의 이미지당 평가 계산이 사라지는 것은 아니며 benchmark의 계산 규칙을 따라야 한다.

CFG는 `teacher/cfg.py`에 공통 구현한다. Low-precision 예측은 CFG 산술 전에 FP32로 승격한다. Student는 학습 중 CFG=2 동작을 흡수하므로 추론에서 unconditional branch를 추가하지 않는다.

5 Distillation: teacher의 동작을 student에 전달

5.1 Knowledge distillation의 의미

Teacher의 prediction이나 trajectory로 student 학습 target을 만든다. Label만 배울 때와 달리 teacher의 구체적인 동작을 모방한다. Parameter 감소뿐 아니라 sampling 계산 감소도 목적이 될 수 있다. 현재는 **같은 크기의 backbone으로 생성에 필요한 evaluation 수를 줄이는 것**이 핵심이다.

Teacher는 target 생성 시 frozen이며 backpropagation 대상이 아니다. Student의 parameter만 업데이트한다. Teacher quality가 낮으면 좋은 target을 제공하기 어렵지만, student가 모든 metric에서 teacher보다 나쁠 수밖에 있다는 수학적 정리는 아니다.

5.2 Guidance distillation과 fixed / variable guidance

Guided distillation은 conditional/unconditional prediction을 조합한 teacher의 결과를 student 한 번의 forward로 근사한다 [4]. Fixed guidance에서는 g를 하나로 고정한다.

$$T_{g=2}(x_t, t, c) \quad \longrightarrow \quad S_\theta(x_t, t, c).$$

Variable guidance라면 $S_\theta(x_t, t, c, g)$처럼 scale도 입력해야 한다. Scale embedding과 guidance sampling distribution이 추가로 필요하다. 현재 구현에는 이 입력이 없으므로 inference kwargs에 scale을 넣는 것으로 guidance를 자유롭게 조절할 수 없다.

5.3 현재 guidance-transfer loss

Teacher의 clipped CFG clean estimate를 y_0라고 하면 student의 velocity target은

$$v^\star = \frac{a_t x_t - y_0}{\sigma_t}.$$

Current loss는 다음처럼 구현된다.

$$L_v = \max(a_t^2, \sigma_t^2)\, \|\hat{v}_\theta - v^\star\|^2, \tag{10}$$

$$L_{x_0} = \max(\mathrm{SNR}(t), 1)\, \|\hat{x}_{0,\theta} - y_0\|^2. \tag{11}$$

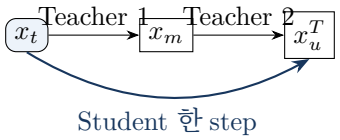
$\hat{x}_{0,\theta} - y_0 = -\sigma_t(\hat{v}_\theta - v^\star)$이므로 두 식은 같다. 큰 SNR을 그대로 velocity loss에 곱하는 것이 아니다. 평균은 batch/channel/pixel에 대해 계산한다.

Guidance transfer는 32-step single-forward student를 먼저 만든다. Holdout FID가 teacher32 FID+15를 넘으면 progressive stages를 진행하지 않는 quality gate가 있다. 이는 prototype 실패 감지 기준이고 benchmark 성공 기준은 아니다.

6 Progressive distillation: 두 step을 한 step으로

6.1 직관과 stage 구성

Teacher가 $t \to m \to u$로 이동한 최종 위치를 student가 $t \to u$ 한 번에 재현하도록 학습한다 [5]. 학습된 student를 다음 stage의 frozen teacher로 사용한다.



현재 stage는 $32 \to 16 \to 8 \to 4 \to 2 \to 1$이다. 최초 halving stage별 25K updates이며 n1/n2 budget extension은 100K까지 완료했다. n1과 n4는 별도 학습 weight다.

6.2 Merged target 유도

식 (8)을 정리하면 student 한 step의 출력은

$$x_u = \frac{\sigma_u}{\sigma_t} x_t + \left(a_u - \frac{\sigma_u}{\sigma_t} a_t\right) \hat{x}_0.$$

이를 teacher 두-step endpoint x_u^T와 같게 두고 풀면

$$\boxed{y_0^{\text{merge}} = \frac{x_u^T - (\sigma_u/\sigma_t)x_t}{a_u - (\sigma_u/\sigma_t)a_t}}. \tag{12}$$

코드의 `merged_target()`가 이 계산을 수행한다. 이어 $v^\star = (a_t x_t - y_0^{\text{merge}})/\sigma_t$로 바꿔 student를 학습한다. 마지막 endpoint가 clean이면 $a_u = 1, \sigma_u = 0$이라 $y_0^{\text{merge}} = x_u^T$다.

Target은 두-step 이동을 한 번에 표현하기 위한 **effective clean prediction**이므로 실제 pixel 범위를 벗어날 수 있다. 이 값을 clip하면 위 식에서 endpoint가 달라진다. 최종 RGB clipping과 intermediate target clipping을 구분해야 한다.

6.3 왜 마지막 2 → 1이 어렵나

한 번의 forward가 noise에서 class structure와 세부 특징까지 모두 생성해야 한다. 이전 stage의 근사 오차와 마지막 압축 난도가 함께 영향을 줄 수 있다. 현재 high-noise node는 $t = 999$에서 $a_t \approx 0.00635$로 0이 아니지만 최고 node의 입력은 pure noise다. 이는 검증할 mismatch 가설이며 확인된 실패 원인은 아니다.

기존 n2 → n1 development FID 증가가 약 16.47로 가장 컸다. n1을 100K까지 연장하면 개선되지만 격차는 남는다. Improved n2 75K로 **새 n1**을 학습하는 것은 기존 n1 continuation과 다른 실험이다.

7 Consistency models와 consistency distillation

7.1 현재 구현과의 경계

> **이 절은 향후 방법의 개념 설명이다.** 현재 code ZIP에는 consistency loss, boundary parameterization, consistency sampling 구현이 없다. Progressive student의 이름을 consistency model로 바꾸는 것으로 구현이 완료되지는 않는다.

Consistency function은 동일한 probability-flow trajectory의 서로 다른 noise-level 점을 같은 low-noise endpoint로 보낸다 [6].

$$f_\theta(x_t, t) \approx f_\theta(x_u, u), \qquad t > u.$$

같은 trajectory라는 조건이 중요하다. 서로 다른 실제 이미지의 출력까지 같게 만드는 loss라면 생성 다양성을 잃는 collapse가 생길 수 있다.

7.2 Score와 probability-flow ODE

Score는 분포 log-density의 입력 gradient, $\nabla_x \log p_t(x)$다. 큰 확률 영역으로 향하는 국소 방향을 나타낸다. Gaussian perturbation의 이상적 epsilon predictor와는 score $\approx -\hat{\epsilon}/\sigma_t$ 관계를 갖는다.

Forward SDE를 $dx = b(x,t)dt + \gamma(t)dW$라고 쓰면 probability-flow ODE는

$$\frac{dx}{dt} = b(x,t) - \frac{1}{2}\gamma(t)^2 \nabla_x \log p_t(x).$$

같은 시간별 marginal distribution을 갖는 결정적 dynamics다. Deterministic DDIM과 관계가 있지만 임의의 discrete DDIM update를 모든 continuous ODE 설정과 동일하다고 간주하면 안 된다.

7.3 Adjacent-pair loss와 boundary condition

Teacher solver로 x_t에서 $\tilde{x}_u$를 만든 뒤 다음과 같은 loss를 사용할 수 있다.

$$L_{\text{CD}} = \mathbb{E}\big[\lambda(t)\, d(f_\theta(x_t, t), \text{sg}[f_{\theta^-}(\tilde{x}_u, u)])\big].$$

sg는 stop-gradient, θ^-는 target network parameter다. Metric d, weighting λ, solver, grid와 target update 정책은 구현에서 명확히 정해야 한다. 일반적인 boundary construction은

$$f_\theta(x,t) = c_{\text{skip}}(t)x + c_{\text{out}}(t)F_\theta(x,t), \quad c_{\text{skip}}(t_{\min}) = 1,\ c_{\text{out}}(t_{\min}) = 0.$$

Low-noise endpoint에서 identity를 강제한다. 본 식은 개념적 형태이며 현재 discrete VP schedule에 그대로 대입할 완성 config는 아니다.

7.4 방법 비교

방법	무엇을 일치시키나	현재 상태
Guidance transfer	Teacher CFG prediction과 student prediction	구현됨
Progressive distillation	Teacher 두 transition과 student 한 transition의 endpoint	구현됨
Consistency distillation	같은 trajectory 점들의 low-noise mapping	미구현
Consistency training	Pretrained teacher 없이 consistency 학습	프로젝트 미구현

Consistency는 one-step을 직접 지원하며 few-step 구성도 가능하지만, 현재 n4 DDIM loop와 동일한 sampling이라고 가정하면 안 된다. Guided trajectory를 사용할 경우 class 및 guidance condition을 두 branch의 consistency 계산에서 맞춰야 한다.

8 U-Net과 conditioning: 수식이 실행되는 구조

8.1 Encoder, decoder, skip connection

U-Net은 downsampling으로 큰 공간 구조를 파악하고 upsampling으로 세부 위치를 복원한다. Encoder feature를 decoder에 concatenate하는 skip connection이 공간 정보를 전달한다. Residual block의 addition shortcut과 encoder–decoder concatenate skip은 다른 연결이다.

구성	현재 구조
Input / output	3 channels, 64×64 RGB
해상도 / channels	64 / 96 → 32 / 192 → 16 / 288 → 8 / 384
Residual / attention	Residual 22개; single-head attention 6개
Attention 위치	Encoder/decoder 16×16, middle 8×8
Condition vector	384-D timestep MLP + class embedding
Parameters	48,883,107; teacher / student 공통 구조

8.2 Residual block, normalization, attention

Residual block은 $y = x + F(x)$ 형태이며 channel이 다르면 shortcut에 projection을 사용한다. GroupNorm은 channel group 단위로 feature를 normalize한다. Condition vector에서 scale과 shift를 만들어

$$h' = \text{GroupNorm}(h) \odot (1 + \text{scale}) + \text{shift}$$

형태로 class/time 정보를 주입한다. Attention은 떨어진 pixel 위치의 feature를 상호 참조해 전체 구조를 조정한다. Query/key/value는 feature projection이며 image classifier를 별도로 부르는 것이 아니다.

8.3 Embedding과 parameter count

Timestep은 sinusoidal feature를 MLP로 변환하고 class는 learned lookup embedding으로 변환한다. 두 vector를 더해 모든 residual block에 전달한다. Class embedding은 pretrained semantic vector가 아니다.

Parameter count는 학습 가능한 weight뿐 아니라 inference에서 쓰는 frozen learned weight도 포함한다. Buffer는 fixed schedule 같은 비학습 tensor다. EMA weight를 선택한 후 live model 하나만 로딩하면 EMA 사본을 추론 parameter로 두 번 더하지 않는다. LoRA는 작은 adaptation matrix를 학습하지만 원래 backbone이 inference에 남으므로 total parameter가 rank만큼 작아지는 것은 아니다. 현재 LoRA는 미구현이다.

9 학습 안정성과 재현성에 필요한 개념

9.1 Optimizer, learning rate, update budget

AdamW는 gradient의 이동 통계를 이용해 parameter를 갱신한다. 현재 weight decay는 0이다. LR은 update 크기이고 noise schedule은 데이터에 더할 noise 비율이다. 새 학습 정책은 constant LR 2×10^{-4}, warmup 없음이다. 과거 decay config는 과거 실험 재현용으로 남는다.

Batch는 update당 이미지 수, gradient accumulation은 여러 microbatch gradient를 모으는 방식이다. 현재 batch 64, accumulation 1이다. 100K는 optimizer updates이며 sampling timesteps 1,000과 다르다. 이미지 노출량은 대략 batch × updates지만 drop-last 등 실제 loader 기록으로 확인해야 한다.

9.2 EMA: weight의 지수 이동평균

$$\theta_{\text{EMA}} \leftarrow \rho\theta_{\text{EMA}} + (1-\rho)\theta.$$

EMA는 parameter 흔들림을 완화하는 weight 평균이다. ρ가 클수록 오래된 weight가 더 남으며 대략 $1/(1-\rho)$ updates의 시간 규모를 갖는다. Teacher primary EMA는 0.9999, student는 0.999다. EMA가 항상 raw보다 좋은 것은 아니므로 selection policy를 고정한다. Consistency target의 EMA 역할과 최종 inference weight 선택 역할도 구분한다.

9.3 AMP, bf16, FP32, clipping, VRAM

Mixed precision은 backbone 일부 계산에 bf16 등의 낮은 precision을 써 속도·메모리를 개선한다. FP32는 precision이 민감한 CFG subtraction, schedule 산술, loss 등에 사용한다. bf16은 FP32보다 mantissa precision이 낮으므로 수치 오차를 없애지 않는다.

Gradient clipping은 큰 gradient norm을 제한하는 것이고 image clipping은 pixel 범위를 제한하는 것이다. Peak allocated VRAM은 tensor가 실제로 할당한 최대 메모리이며 GPU 사용률이나 allocator reserved memory와 다르다. 현재 budget 실험 측정치는 약 9.80 GB, 즉 약 9.12 GiB다. GB=10^9 bytes, GiB=2^{30} bytes이다.

9.4 Checkpoint, export, resume

종류	저장 내용	용도
Full training checkpoint	Student/raw weight, EMA, optimizer, RNG, cursor, config 등	정확한 학습 continuation
Inference checkpoint	선택 EMA의 state dict, schedule buffer, provenance 등	제출 / 이미지 생성

Inference checkpoint에서 optimizer를 새로 만들면 weight fine-tuning은 가능하지만 원래 학습의 exact resume는 아니다. Seed뿐 아니라 RNG state와 데이터 위치가 필요하다. 같은 explicit sampling seed의 재현성과 다른 hardware/precision에서의 byte-identical 재현은 별개의 주장이다.

10 NFE, FID, holdout: 결과를 올바르게 읽기

10.1 Steps와 NFE의 차이

Steps는 sampler transition 수, NFE(number of function evaluations)는 backbone 평가 수다. Current plain student는 step당 한 번, CFG=2 teacher는 두 번 평가한다.

모델	Steps	NFE	설명
Teacher DDIM32 CFG2	32	64	Cond + uncond
Guidance student	32	32	CFG 동작 흡수
Few-NFE student	4	4	별도 n4 weight
One-NFE student	1	1	별도 n1 weight

NFE가 작아도 latency가 정확히 비례하는 것은 아니다. Backbone 크기, batch, device transfer, precision과 실행 overhead도 영향을 준다. NFE는 실제 forward hook 등으로 확인해야 한다.

10.2 FID의 의미와 한계

FID는 pretrained Inception feature의 평균과 covariance를 이용한 분포 차이다 [7].

$$\mathrm{FID} = \|\mu_r - \mu_g\|_2^2 + \mathrm{Tr}\Big(\Sigma_r + \Sigma_g - 2(\Sigma_r^{1/2}\Sigma_g\Sigma_r^{1/2})^{1/2}\Big).$$

낮을수록 두 feature 분포가 가깝다. r은 real reference, g는 generated set이다. FID는 pixel-wise reconstruction error도, 개별 이미지의 정확도도 아니다. 같은 preprocessing, feature extractor, reference, sample count와 seed 정책으로 비교해야 한다. 작은 reference set과 단일 seed는 통계 변동성을 갖는다.

FID 하나로 class correctness나 다양성의 원인을 전부 설명할 수 없다. Class별 이미지 검토와 quality/coverage 분석이 보완책이다. Training에 pretrained 생성 모델을 쓰지 않는 규칙과 평가용 Inception feature extractor는 구분한다.

10.3 Training-only holdout과 official validation

데이터	이미지 수	역할
공식 training 전체	17,079	내부 분리의 원본
Optimizer training subset	16,324	Gradient update
Training-only holdout	755	Teacher / config / checkpoint 선택
공식 validation	3,020	고정 설정의 공식 성능 보고

Holdout은 class당 5장, seed 42로 고정했다. Official validation으로 학습하거나 반복 튜닝하면 leakage가 된다. Development와 official FID는 reference가 다르므로 숫자의 대소로 일반화 성능을 판정하지 않는다.

10.4 Ablation과 seed

Ablation은 하나의 변경 원인을 비교하는 통제 실험이다. n1 25K → 100K는 teacher를 고정해 budget 영향을 확인했다. Improved n2로 새 n1을 학습하면 teacher target 자체가 바뀐다. Sampling seed 반복은 생성 변동성을, training seed 반복은 학습 변동성을 측정한다. 여러 sampling seed를 여러 training run처럼 해석하면 안 된다.

11 개념을 현재 프로젝트에 연결하기

11.1 최신 결과와 현재 병목

다음은 2026-09-30 완료 기록의 보고값이다. 두 FID column은 서로 다른 reference를 사용한다.

모델	NFE	Dev FID	Official FID
Selected noaug teacher, 100K	64	44.11891	26.65365
Guidance-transfer student, 50K	32	43.25620	—
Progressive n4, 25K	4	47.62077	33.0517
기존 n1, 25K	1	68.16127	61.4540
개선 n1, 100K	1	65.38627	58.1595
Budget n2, 75K 선택	2	50.89945	39.54904

Inference ZIP의 one-NFE는 개선된 100K weight, few-NFE는 기존 n4 weight다. 각각 48,883,107 parameters다. n1 extension은 기존 25K n2를 teacher로 유지했다. Improved n2를 이용한 새 n1은 아직 완료 결과가 없다.

11.2 각 실험의 질문과 연결 관계

1. **Noaug teacher 100K:** 원본 train subset으로 augmentation 없이 diffusion teacher를 학습했다. Holdout로 선택한 teacher를 DDIM32 CFG2로 보고한다. Cond/uncond 두 평가를 32번 반복해 64 NFE다. 이후 student의 품질 기준점이다.
2. **Fixed guidance student 50K:** Teacher CFG=2의 동작을 student 하나의 conditional forward에 담는다. Transition은 32번 유지하고 branch 계산만 줄여 64에서 32 NFE가 된다. Official FID는 별도 보고되지 않았다.
3. **Progressive n4 25K:** Guidance student에서 32→16→8→4로 압축했다. 25K는 n4 단계 자체의 budget이며 앞 단계의 학습 비용은 별도다. 현재 few-NFE 제출 weight다.
4. **기존 n1 25K:** 4→2→1까지 이어 마지막 두 teacher transitions를 한 student transition으로 압축했다. 별도 학습된 one-step baseline이며 n4 loop만 줄인 모델은 아니다.
5. **Budget n1 100K:** 기존 n1의 optimizer/EMA/RNG를 보존해 75K를 추가했다. Teacher는 기존 25K n2로 고정했다. Budget만 늘린 비교로 official FID가 61.4540에서 58.1595로 개선됐다.
6. **Budget n2 75K 선택:** 기존 n4를 teacher로 고정하고 n2를 25K에서 100K까지 연장했다. Holdout가 가장 좋은 75K weight를 선택했다. 이 개선 n2로 새 n1을 학습하는 것은 아직 미실행이다.

마지막 두 continuation은 독립적인 budget 비교다. 개선 n1이 개선 n2의 target으로 학습됐다는 계보는 성립하지 않는다.

11.3 코드 찾아보기

파일 / 함수	연결되는 개념
`teacher/model.py`	U-Net, embeddings, residual / attention
`teacher/diffusion.py`	Linear noise schedule, epsilon loss, DDPM/DDIM
`teacher/cfg.py`	Classifier-free guidance의 공통 식
`distillation/core.py`	Coefficients, velocity, DDIM step, merged target, loss
`distillation/train.py`	Frozen teacher와 student training
`distillation/pipeline.py`	Teacher selection, quality gate, stage sequencing
`distillation/submission.py`	1/4-step grid, sampling buffer, seed
`distillation/export.py`	EMA export와 provenance
`model.py`	ModelOneNFE / ModelFewNFE 제출 interface
`evaluate.py`	Upstream의 보호된 official evaluator

11.4 One-NFE 한 번의 생성 따라가기

1. Class ID를 입력하고 seed로 Gaussian x_{999}를 생성한다.
2. Backbone을 한 번 호출해 $\hat{v}(x_{999}, 999, c)$를 얻는다.
3. $\hat{x}_0 = a_{999}x_{999} - \sigma_{999}\hat{v}$로 clean estimate를 구한다.
4. Exact clean endpoint -1로 DDIM update를 수행하고 최종 RGB를 clip한다.

이 과정에는 teacher, unconditional CFG branch, 별도 learned decoder가 없다. Training timesteps가 1,000이어도 inference forward는 하나다.

11.5 다음 연구를 판단하는 질문

개선된 n2 target으로 새 n1을 학습할 때는 student budget, split, LR와 평가 조건을 맞춰 비교한다. Consistency를 구현할 때는 boundary condition, target update, solver, grid와 1/4-NFE 실제 호출을 독립 검증한다. Schedule mismatch는 확인할 가설이므로 해결되었다고 먼저 결론 내리지 않는다.

12 빠른 용어 사전과 자주 생기는 혼동

용어	한 문장 정리
Backbone	실제 prediction을 계산하는 learned network.
Conditioning	Time/class 등 추가 정보를 network 동작에 반영하는 것.
Null token	조건을 제거한 branch를 학습하기 위한 특수 embedding ID.
CFG	Conditional/unconditional prediction 차이를 scale로 강조하는 것.
Distillation	Teacher의 동작을 student target으로 전달하는 학습.
Guidance transfer	CFG의 다중 branch 동작을 single-forward student에 전달.
Progressive	Teacher 두 transition을 student 한 transition으로 반복 압축.
Consistency	같은 trajectory의 점들을 같은 endpoint로 매핑하는 성질.
Trajectory	Noise에서 clean 쪽으로 이동하는 states의 경로.
SNR	신호 분산 / noise 분산 비율.
EMA	Parameter의 지수 이동평균.
NFE	Sampling 중 이미지당 backbone 평가 횟수.
FID	Inception feature 분포의 평균·covariance 차이.
Holdout	Optimizer에서 제외하고 개발 판단에 사용하는 데이터.
Ablation	영향을 분리하기 위한 통제 비교.
Provenance	Weight가 어떤 teacher·설정·checkpoint에서 왔는지의 기록.
LoRA	Low-rank adaptation; 현재 연구의 optional extension.

혼동 방지

- CFG가 classifier-free라고 해서 class condition이 없는 것이 아니다.
- Teacher/student architecture가 같아도 distillation이 가능하다.
- 1-NFE checkpoint는 4-NFE checkpoint의 loop만 줄인 모델이 아니다.
- EMA는 optimizer도 sampler도 아니며 weight 평균이다.
- 낮은 epsilon loss가 낮은 FID를 보장하지 않는다.
- 100K updates와 1,000 timesteps, 32 sampling steps는 서로 다른 숫자다.
- Seed 일치는 sampling 조건 일부의 일치이며 모든 run 독립성을 보장하지 않는다.
- Consistency 목표를 적은 기획 문서가 있어도 현재 구현은 progressive다.

12.1 이해 확인 문제

1. DDIM32 CFG=2 teacher와 32-step guidance student의 NFE는? **64와 32.**
2. $\epsilon_u = 0.1, \epsilon_c = 0.4, g = 2$면 guided prediction은? **0.7.**
3. Student가 v를 예측할 때 clean estimate는? $a_t x_t - \sigma_t \hat{v}$**.**
4. Merged target을 clip하면 문제가 생길 수 있는 이유는? **두-step endpoint 일치 식이 바뀐다.**
5. Inference checkpoint로 정확한 학습 continuation이 가능한가? **Optimizer·RNG 등 full state가 없어 불가능하다.**
6. Development FID와 official FID가 다른 주된 조건은? **Reference set이 다르다.**

13 출처와 재생성 방법

13.1 프로젝트의 1차 근거

- `TEAM_HANDOFF.md`: 전달 내용, 정확한 base commit, latest inference checkpoint.
- Code ZIP의 `docs/distillation_baseline.md`: 방법, v-loss, grid, initialization, smoke 기록.
- `docs/distillation_results_review.md`: 최초 완료 FID와 1-NFE 진단.
- `docs/low_nfe_budget_experiment.md`: 100K 완료 결과와 선택 checkpoint.
- `docs/model_architecture.md` 및 실제 `teacher/model.py`: U-Net 구조. 과거 현황·LR은 최신 config로 보완.
- `distillation/core.py`, `distillation/submission.py`: 이 문서의 merged-target 수식과 실제 추론 흐름.

위 코드 경로는 overlay ZIP 내부 및 복원 checkout의 상대 경로다. 실행용 checkout은 `/root/Diffusion/workspace_distillation`에 복원했다. Base commit은 `e3f5120000cd91037017d86f23993492195101a5`이다. 복원 상세와 축별 완료/개선 지점은 별도 `DIFFUSION_STATUS_AND_EXPERIMENTS.md`에 정리했다.

이번 복원 확인: Python 3.10 / PyTorch 2.6.0+cu124, 실제 data와 GPU에서 1/4-NFE checkpoint를 검증했다. 변경하지 않은 official evaluator로 재계산한 FID는 58.1595 / 33.0517이며 handoff 보고값과 소수점 네 자리까지 일치했다. Teacher와 중간 stage의 full training checkpoint는 없어 정확한 학습 continuation은 별도 artifact가 필요하다.

13.2 LaTeX에서 PDF 생성

XeLaTeX, Korean TeX support, Noto CJK font가 설치된 환경에서 다음 명령을 두 번 실행한다. 두 번째 실행은 목차·인용·수식 참조를 확정한다.

```
cd /root/Diffusion
xelatex -interaction=nonstopmode -halt-on-error DIFFUSION_CONCEPTS.tex
```

이 문서의 일반 개념은 아래 원 논문을 참고하고 프로젝트 설명은 로컬 구현을 근거로 했다. 모든 loss와 sampler 선택을 원 논문 전체의 완전 재현이라고 주장하지 않는다.

References

[1] Ho, Jain, Abbeel. *Denoising Diffusion Probabilistic Models*. 2020. https://arxiv.org/abs/2006.11239.

[2] Song, Meng, Ermon. *Denoising Diffusion Implicit Models*. 2020. https://arxiv.org/abs/2010.02502.

[3] Ho, Salimans. *Classifier-Free Diffusion Guidance*. 2022. https://arxiv.org/abs/2207.12598.

[4] Meng et al. *On Distillation of Guided Diffusion Models*. CVPR 2023. https://arxiv.org/abs/2210.03142.

[5] Salimans, Ho. *Progressive Distillation for Fast Sampling of Diffusion Models*. ICLR 2022. https://arxiv.org/abs/2202.00512.

[6] Song, Dhariwal, Chen, Sutskever. *Consistency Models*. ICML 2023. https://arxiv.org/abs/2303.01469.

[7] Heusel et al. *GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium*. 2017. https://arxiv.org/abs/1706.08500.